DG1IAD, Internet Applications & Database Design

Dr Alexandros Giagkos

Computer Science
College of Engineering and Physical Sciences

Important messages from last week

Aston University BIRMINGHAM UK

lecturer (lec_id, name, .., city)

lec_id	name	..	city
L1	Alex	..	B'ham
..	..	..	..

student (stu_id, name, .., city)

stu_id	name	..	city
S1	Armit	..	B'ham
..	..	..	..

- Relations, or tables, group data about entities
- Entity attributes are found in columns
- Entity instances, or tuples, are found in rows

- Keys uniquely identify tuples and form relationships: one-to-one, one-to-many, many-to-many partial and total participation

Important messages from last week

lecturer (lec_id, name, .., city)

lec_id	name	..	city
L1	Alex	..	B'ham
..	..	..	..

student (stu_id, name, .., city)

stu_id	name	..	city
S1	Armit	..	B'ham
..	..	..	..

Database Development

- Week 1: Relational Databases and SQL
  - Lab: Install XAMPP, and perform CRUD operations
- **Week 2: Functional Dependences and Normal Forms**
  - **Lab: Practising relationships and normalised designs**
- Week 3: Data Integrity, ACID and Transactions
  - Lab: Complete unfinished lab work

Functional Dependences and Normal Forms

Outline:

- Part I: Database anomalies
- Part II: Schema decomposition and SQL Joins
- Part III: Functional dependences
- Part IV: Normal forms (1NF to 3NF)

Learning outcomes:

- Understand what is meant by a good database design
- Perform schema decomposition to reach a good design

Database anomalies

What is a "good" database design?

Aston University
BIRMINGHAM UK

- The goal of database design is to ultimately create a set of relations that allow us to:
  - store data without unnecessary redundancy
  - retrieve data efficiently and easily
- A bad design renders a database prone to anomalies
- We fight data redundancy by applying normalisation

Insertion anomaly:

- Storing incomplete information due to multiple concepts in rows

Consider the following relation of students' data:

LastName	FirstName	Email	Tutor	TutorEmail	Dept
Andrews	Matthew	anm16	Price	cjp	CompSci
Thompson	James	thj16	Price	cjp	CompSci
Chen	Fang	chf16	Wilson	mxw	CompSci
			Shen	qqs	CompSci

- Adding Shen as a new tutor leads to incomplete cells, affecting consistency, integrity and access of the data

Database Anomalies: Update

Aston University
BIRMINGHAM UK

Update anomaly:

- Updating specific values affects data integrity due to duplications

LastName	FirstName	Email	Tutor	TutorEmail	Dept
Andrews	Matthew	anm16	Price	cjp	CompSci
Thompson	James	thj16	Price	cjp	CompSci
Chen	Fang	chf16	Wilson	mxw	CompSci

- Changing tutor for James Thompson requires two different database updates

Database Anomalies: Deletion

Aston University
BIRMINGHAM UK

Deletion anomaly:

- Deleting rows may unintentionally remove important data

LastName	FirstName	Email	Tutor	TutorEmail	Dept
Andrews	Matthew	anm16	Price	cjp	CompSci
Thompson	James	thj16	Price	cjp	CompSci
Chen	Fang	chf16	Wilson	mxw	CompSci

- Removing Fang Chen from the table will also remove his tutor's data

Summary on database anomalies

Aston University
BIRMINGHAM UK

- If the database is not well designed, the overall data integrity will deteriorate over time
- Anomalies occur due to data redundancy and/or lack of proper constraints
- Flat-file databases almost always lead to anomalies
- Breaking flat-file databases into smaller, focused and well-maintained tables is the only cure

Aston University
BIRMINGHAM UK

Schema decomposition and SQL Joins

Universal relation

- The universal relation assumption:
  - All data attributes can be stored into a single table, which can be decomposed into smaller tables (schema decomposition)
  - Then, SQL `SELECT` statements can `JOIN` smaller tables together to retrieve and present data
- Two questions:
  - How do we `JOIN` tables together?
  - How do we decompose a table into smaller ones?

JOIN Expressions

- The `JOIN` operation takes two relations and returns one, given that tuples in the given relations match under some condition
- It is typically used as subquery expression in the `FROM` part of a `SELECT` query

JOIN Expressions - Tutors and Tutees

Aston University BIRMINGHAM UK

- Consider the relations Tutor (left) and Tutee (right)

TUTOR_ID	NAME	DEPT_NAME	OFFICE
1	Thomas Angel	CompSci	MB122A
2	Lorraine Batchelor	CompSci	MB122B
3	Adele Black	Maths	MB111A
4	Murray Brownlie	Mech.Eng.	MB118B
5	John Bullen	CompSci	MB112C

TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
3	Mike Gaffney	4	Mech.Eng.
1	Patricia Currie	1	CompSci
2	Peter Edghill	2	CompSci
4	Michael Gregory	3	Maths
5	Russell Griffiths	-	CompSci
6	Anne Joseph	-	Physics

- Note that there are **tutors who have no tutees**, e.g., John, and **tutees that are not assigned to a tutor**, e.g., Russell and Anne (FK allows NULL or partial participation).

Question 1

- Consider the relations Tutor (left) and Tutee (right)

TUTOR_ID	NAME	DEPT_NAME	OFFICE
1	Thomas Angel	CompSci	MB122A
2	Lorraine Batchelor	CompSci	MB122B
3	Adele Black	Maths	MB111A
4	Murray Brownlie	Mech.Eng.	MB118B
5	John Bullen	CompSci	MB112C

TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
3	Mike Gaffney	4	Mech.Eng.
1	Patricia Currie	1	CompSci
2	Peter Edghill	2	CompSci
4	Michael Gregory	3	Maths
5	Russell Griffiths	-	CompSci
6	Anne Joseph	-	Physics

- We need a query to:

"select all attributes from all tutors and tutees that participate in the relationship"

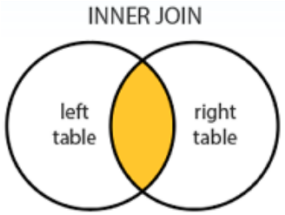
JOIN Expressions - INNER

- Find all matching tuples from two tables:

```
SELECT * FROM tutor INNER JOIN tutee
ON tutor.tutor_id = tutee.tutor_id ORDER BY tutor.tutor_id;
```

TUTOR_ID	NAME	DEPT_NAME	OFFICE	TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
1	Thomas Angel	CompSci	MB122A	1	Patricia Currie	1	CompSci
2	Lorraine Batchelor	CompSci	MB122B	2	Peter Edghill	2	CompSci
3	Adele Black	Maths	MB111A	4	Michael Gregory	3	Maths
4	Murray Brownlie	Mech.Eng.	MB118B	3	Mike Gaffney	4	Mech.Eng.

- In the `JOIN` above, tutor is the left table, and tutee is the right table



Question 2

Aston University
BIRMINGHAM UK

- Consider the relations Tutor (left) and Tutee (right)

TUTOR_ID	NAME	DEPT_NAME	OFFICE
1	Thomas Angel	CompSci	MB122A
2	Lorraine Batchelor	CompSci	MB122B
3	Adele Black	Maths	MB111A
4	Murray Brownlie	Mech.Eng.	MB118B
5	John Bullen	CompSci	MB112C

TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
3	Mike Gaffney	4	Mech.Eng.
1	Patricia Currie	1	CompSci
2	Peter Edghill	2	CompSci
4	Michael Gregory	3	Maths
5	Russell Griffiths	-	CompSci
6	Anne Joseph	-	Physics

- We need a query to:

"select all attributes from all tutors, including their tutees' data, that participate in the relationship, as well as those tutors that don't have tutees yet"

JOIN Expressions - LEFT OUTER

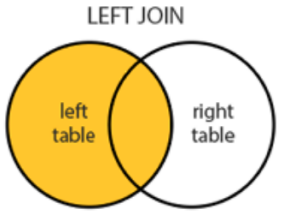
Aston University BIRMINGHAM UK

- Find all matching tuples and those that do not have a match from the left table:

```
SELECT * FROM tutor LEFT OUTER JOIN tutee
ON tutor.tutor_id = tutee.tutor_id ORDER BY tutor.tutor_id;
```

TUTOR_ID	NAME	DEPT_NAME	OFFICE	TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
1	Thomas Angel	CompSci	MB122A	1	Patricia Currie	1	CompSci
2	Lorraine Batchelor	CompSci	MB122B	2	Peter Edghill	2	CompSci
3	Adele Black	Maths	MB111A	4	Michael Gregory	3	Maths
4	Murray Brownlie	Mech.Eng.	MB118B	3	Mike Gaffney	4	Mech.Eng.
5	John Bullen	CompSci	MB112C	-	-	-	-

- Note that `LEFT OUTER` and `LEFT` is the same



Question 3

Aston University
BIRMINGHAM UK

- Consider the relations Tutor (left) and Tutee (right)

TUTOR_ID	NAME	DEPT_NAME	OFFICE
1	Thomas Angel	CompSci	MB122A
2	Lorraine Batchelor	CompSci	MB122B
3	Adele Black	Maths	MB111A
4	Murray Brownlie	Mech.Eng.	MB118B
5	John Bullen	CompSci	MB112C

TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
3	Mike Gaffney	4	Mech.Eng.
1	Patricia Currie	1	CompSci
2	Peter Edghill	2	CompSci
4	Michael Gregory	3	Maths
5	Russell Griffiths	-	CompSci
6	Anne Joseph	-	Physics

- We need a query to:

"select all attributes from all tutors, including their tutees' data, that participate in the relationship, as well as those tutees that don't have a tutor yet"

JOIN Expressions - RIGHT OUTER

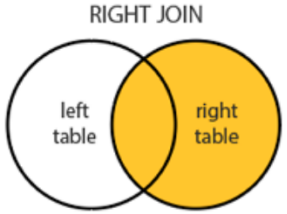
Aston University BIRMINGHAM UK

- Find all matching tuples and those that do not have a match from the right table:

```
SELECT * FROM tutor RIGHT OUTER JOIN tutee
ON tutor.tutor_id = tutee.tutor_id ORDER BY tutor.tutor_id;
```

TUTOR_ID	NAME	DEPT_NAME	OFFICE	TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
1	Thomas Angel	CompSci	MB122A	1	Patricia Currie	1	CompSci
2	Lorraine Batchelor	CompSci	MB122B	2	Peter Edghill	2	CompSci
3	Adele Black	Maths	MB111A	4	Michael Gregory	3	Maths
4	Murray Brownlie	Mech.Eng.	MB118B	3	Mike Gaffney	4	Mech.Eng.
-	-	-	-	5	Russell Griffiths	-	CompSci
-	-	-	-	6	Anne Joseph	-	Physics

- Note that `RIGHT OUTER` and `RIGHT` is the same



Question 4

- Consider the relations Tutor (left) and Tutee (right)

TUTOR_ID	NAME	DEPT_NAME	OFFICE
1	Thomas Angel	CompSci	MB122A
2	Lorraine Batchelor	CompSci	MB122B
3	Adele Black	Maths	MB111A
4	Murray Brownlie	Mech.Eng.	MB118B
5	John Bullen	CompSci	MB112C

TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
3	Mike Gaffney	4	Mech.Eng.
1	Patricia Currie	1	CompSci
2	Peter Edghill	2	CompSci
4	Michael Gregory	3	Maths
5	Russell Griffiths	-	CompSci
6	Anne Joseph	-	Physics

- We need a query to:

"select all attributes from all tutors and tutees, matching those that participate in the relationship, but also including all others that don't"

JOIN Expressions - FULL (OUTER)

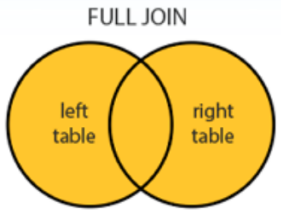
Aston University BIRMINGHAM UK

- Find all matching tuples and those that have no match from both left and right tables:

```
SELECT * FROM tutor FULL JOIN tutee
ON tutor.tutor_id = tutee.tutor_id ORDER BY tutor.tutor_id;
```

TUTOR_ID	NAME	DEPT_NAME	OFFICE	TUTEE_ID	NAME	TUTOR_ID	DEPT_NAME
1	Thomas Angel	CompSci	MB122A	1	Patricia Currie	1	CompSci
2	Lorraine Batchelor	CompSci	MB122B	2	Peter Edghill	2	CompSci
3	Adele Black	Maths	MB111A	4	Michael Gregory	3	Maths
4	Murray Brownlie	Mech.Eng.	MB118B	3	Mike Gaffney	4	Mech.Eng.
5	John Bullen	CompSci	MB112C	-	-	-	-
-	-	-	-	5	Russell Griffiths	-	CompSci
-	-	-	-	6	Anne Joseph	-	Physics

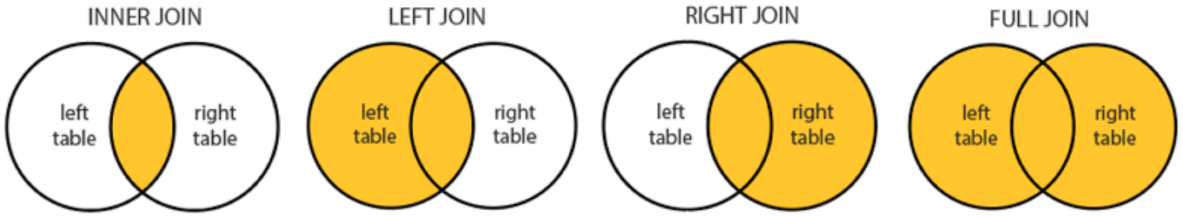
- Combines `RIGHT` and `LEFT OUTER JOIN`



JOIN Expressions - General Form

- Given all four types of `JOIN`, one can use the following syntax:

```
SELECT column1, column2..
FROM left_table
type JOIN right_table
ON (matching criterion)
WHERE condition1, condition2..
ORDER BY value
```



Decomposition and JOINs

- Efficiently decomposing large tables allows us to:
  - reduce data redundancy,
  - improve integrity and
  - keep a clean database which can "answer" the application's queries
- JOINs are effective iff primary keys are chosen correctly
- Our database design must lead to **lossless joins**

  Ironically, with a bad design, the RDBMS will match rows from respective relations incorrectly producing *more* data that do not reflect what we expect/have stored.

Lossless Joins (1/2)

Aston University
BIRMINGHAM UK

PERSON_ID	NAME	STREET	CITY	SALARY
100	Nick	North	Manchester	110000
110	Myra	Main	Liverpool	19000
120	Nick	Old	Coventry	27000
130	Ross	South	Carmarthen	91000
140	Alex	North	Edinburgh	80000
150	David	Back	Birmingham	45000

PK

PERSON_ID	NAME
100	Nick
110	Myra
120	Nick
130	Ross
140	Alex
150	David

FK

NAME	STREET	CITY	SALARY
Nick	North	Manchester	110000
Myra	Main	Liverpool	19000
Nick	Old	Coventry	27000
Ross	South	Carmarthen	91000
Alex	North	Edinburgh	80000
David	Back	Birmingham	45000

PERSON_ID	NAME	STREET	CITY	SALARY
100	Nick	North	Manchester	110000
120	Nick	North	Manchester	110000
110	Myra	Main	Liverpool	19000
100	Nick	Old	Coventry	27000
120	Nick	Old	Coventry	27000
130	Ross	South	Carmarthen	91000
140	Alex	North	Edinburgh	80000
150	David	Back	Birmingham	45000

Lossy join decomposition — integrity lost with name as superkey!

FD name $\rightarrow$ person_id, street, city, salary

Lossless Joins (2/2)

Aston University
BIRMINGHAM UK

PERSON_ID	NAME	STREET	CITY	SALARY
100	Nick	North	Manchester	110000
110	Myra	Main	Liverpool	19000
120	Nick	Old	Coventry	27000
130	Ross	South	Carmarthen	91000
140	Alex	North	Edinburgh	80000
150	David	Back	Birmingham	45000

PK

PERSON_ID	NAME
100	Nick
110	Myra
120	Nick
130	Ross
140	Alex
150	David

FK

NAME	STREET	CITY	SALARY
Nick	North	Manchester	110000
Myra	Main	Liverpool	19000
Nick	Old	Coventry	27000
Ross	South	Carmarthen	91000
Alex	North	Edinburgh	80000
David	Back	Birmingham	45000

PERSON_ID	NAME	STREET	CITY	SALARY
100	Nick	North	Manchester	110000
120	Nick	North	Manchester	110000
110	Myra	Main	Liverpool	19000
100	Nick	Old	Coventry	27000
120	Nick	Old	Coventry	27000
130	Ross	South	Carmarthen	91000
140	Alex	North	Edinburgh	80000
150	David	Back	Birmingham	45000

Lossy join decomposition — integrity lost with name as superkey!

FD name $\rightarrow$ person_id, street, city, salary

Lossless Joins (cont.)

Aston University
BIRMINGHAM UK

PERSON_ID	NAME	STREET	CITY	SALARY
100	Nick	North	Manchester	110000
110	Myra	Main	Liverpool	19000
120	Nick	Old	Coventry	27000
130	Ross	South	Carmarthen	91000
140	Alex	North	Edinburgh	80000
150	David	Back	Birmingham	45000

PK

PERSON_ID	NAME
100	Nick
110	Myra
120	Nick
130	Ross
140	Alex
150	David

FK

PERSON_ID	STREET	CITY	SALARY
100	North	Manchester	110000
110	Main	Liverpool	19000
120	Old	Coventry	27000
130	South	Carmarthen	91000
140	North	Edinburgh	80000
150	Back	Birmingham	45000

PERSON_ID	NAME	STREET	CITY	SALARY
100	Nick	North	Manchester	110000
110	Myra	Main	Liverpool	19000
120	Nick	Old	Coventry	27000
130	Ross	South	Carmarthen	91000
140	Alex	North	Edinburgh	80000
150	David	Back	Birmingham	45000

Lossless join decomposition — integrity kept (superkey person_id)

FD person_id $\rightarrow$ name, street, city, salary

Functional dependencies

It's all about the keys!

Aston University
BIRMINGHAM UK

- We need to apply some theory to help us identify:
  1. superkeys of a relation
  2. whether it is OK to decompose it into smaller relations
     - ensuring the recreation of the original relation through `JOIN`
     - evaluating the decomposed schema
- We seek to enforce wanted dependencies, and eliminate unwanted ones:

  "If I know the *name* I can figure out the *city*"

  (see last example)

Functional Dependencies

Aston University BIRMINGHAM UK

- A functional dependency (FD) is an integral part of the relational model — a form of constraint
- FD $\alpha \rightarrow \beta \implies (t_1[\alpha] = t_2[\alpha] \implies t_1[\beta] = t_2[\beta])$
  - Determinant α functionally determines dependent β
  - If two tuples t_1 and t_2 agree on the α attribute, then they must agree on the β attribute too

TUTOR_ID	NAME	DEPT_NAME	OFFICE
1	Thomas Angel	CompSci	MB122A
2	Lorraine Batchelor	CompSci	MB122B
3	Adele Black	Maths	MB111A
4	Murray Brownlie	Mech.Eng.	MB118B
5	John Bullen	CompSci	MB112C

☺ FD tutor_id $\rightarrow$ name

☹ FD dept_name $\rightarrow$ name

Determining Functional Dependencies

Aston University
BIRMINGHAM UK

- FDs ensure data integrity and no data redundancy in a database
- For any relation, the initial set of FDs are given by the project specification (common sense, experience and business-related)
- Further FDs can then be inferred by applying axioms & rules

  (See Chapter 7, Silberschatz, A., Korth, FH., and Sudarshan, S., Database System Concepts, McGraw-Hill, 2020)
- In general, we aim to have fully functional dependencies and eliminate partial and transitive dependencies as they cause design anomalies

Fully Functional Dependency

- Definition: FD $\alpha \rightarrow \beta$ is a fully functional dependency if β is full functionally dependent on α, **but not** on any proper subset of α

  given: $\{$supplier_id, item_id $\} \rightarrow$ price

supplier_id (PK)	**item_id** (PK)	**price**
20	4	5.99
21	4	9.99
20	2	3.80
25	8	8.45

  You need to know both supplier_id and item_id to determine the price

Partial Dependency

- Definition: FD $\alpha \rightarrow \beta$ is a partial dependency if β is functionally dependent on α, and β **can be** determined by a proper subset of α

  given: $\{$ student_id, project_id $\} \rightarrow$ student_name

student_id (PK)	**project_id** (PK)	**student_name**	**project_title**
AU45	5	Ellie Jones	FlySim
AU99	14	Ben Roberts	NetPuzzle

- **BUT** the student_name and project_title functionally depend on part of the candidate key student_id, project_id, respectively

Transitive Dependency

- Definition:
  If FD $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$ hold, then $\alpha \rightarrow \gamma$ (axiom of transitivity)

  given: class $\rightarrow$ teacher_name

  but from the data: teacher_name $\rightarrow$ teacher_age

  then: class $\rightarrow$ teacher_age **!!!**

class (PK)	**teacher_name**	**teacher_age**
A1	Jack	42
A9	Mary	29

- With class being the candidate key, the non-key attribute teacher_age depend on the non-key attribute teacher_name

Aston University
BIRMINGHAM UK

Normal forms (1NF to 3NF)

What is normalisation?

- Normalisation is a set of criteria for eliminating redundant data and avoiding operation anomalies
- Become more restrictive as you go down the list:
  - **1NF**: all tables are flat
  - **2NF**: good enough
  - **3NF**: most common
- Hierarchy is followed, e.g., if your schema complies with 3NF, it should comply with 2NF and 1NF

First Normal Form (1NF)

- All types must have atomic domains
- No repeated groups

Student

id (PK)	fname	sname	nextofkin
5	Stephen	Garrett	(M. Garrett, K. Garrett)
7	Lucy	Warren	B. Warren
9	Myra	Gordon	C. Gordon
4	Neil	Davis	L. Davis
6	David	Binder	A. Binder

- What is wrong here?

First Normal Form (1NF)

- All types must have atomic domains
- No repeated groups

Student

id (PK)	fname	sname	nextofkin1	nextofkin2
5	Stephen	Garrett	M. Garrett	K. Garrett
7	Lucy	Warren	B. Warren	
9	Myra	Gordon	C. Gordon	
4	Neil	Davis	L. Davis	
6	David	Binder	A. Binder	

- Again, what is wrong here?

First Normal Form (1NF)

- All types must have atomic domains
- No repeated groups

Student

id (PK)	fname	sname	nextofkin
5	Stephen	Garrett	M. Garrett
5	Stephen	Garrett	K. Garrett
7	Lucy	Warren	B. Warren
9	Myra	Gordon	C. Gordon
4	Neil	Davis	L. Davis
6	David	Binder	A. Binder

- Complies with 1NF ☺ — any issues?

Second Normal Form (2NF)

- All relations must comply with 1NF
- Non-key attributes must fully depend on the candidate keys

Student

id (PK)	fname	sname	nextofkin (PK)	nextofkin_tel
5	Stephen	Garrett	M. Garrett	255-0608
5	Stephen	Garrett	K. Garrett	739-6134
7	Lucy	Warren	B. Warren	999-6173
9	Myra	Gordon	C. Gordon	248-9612
4	Neil	Davis	L. Davis	287-6479
6	David	Binder	A. Binder	644-2297

- Note that now $\{$id, nextofkin$\}$ constitutes the PK
  A **non-key attribute** is determined by a part of it
- Is the above database in 2NF?

Second Normal Form (2NF)

Aston University
BIRMINGHAM UK

- All relations must comply with 1NF
- Non-key attributes must fully depend on the candidate keys

Student

id (PK)	fname	sname
5	Stephen	Garrett
7	Lucy	Warren
9	Myra	Gordon
4	Neil	Davis
6	David	Binder

Relatives

nextofkin (PK)	nextofkin_tel	stu_id (FK)
M. Garrett	255-0608	5
K. Garrett	739-6134	5
B. Warren	999-6173	7
C. Gordon	248-9612	9
L. Davis	287-6479	4
A. Binder	644-2297	6

- Complies with 2NF ☺ — and is "good enough"

Discussion on 2NF

Aston University BIRMINGHAM UK

- Given that all 1NF properties are met for a relation, if it has only simple candidate keys, then it is also in 2NF
  - No PDs with simple candidate keys

Relatives (nextofkin, nextofkin_tel, building, postcode, stu_id)

nextofkin (PK)	nextofkin_tel	building	postcode	stu_id (FK)
M. Garrett	255-0608	Wembley	SY23 5DD	5
K. Garrett	739-6134	Wembley	SY23 5DD	5
B. Warren	999-6173	Jendale	HU7 4ET	7
C. Gordon	248-9612	Ifton	SY11 3AF	9
L. Davis	287-6479	Vega	WD23 4TW	4
A. Binder	644-2297	Maffon	CF46 6HE	6

- building $\rightarrow$ postcode leads to anomalies
- Transitive dependency (TD): **a non-key attribute determining a non-key attribute**

Remember $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma \implies \alpha \rightarrow \gamma$

Third Normal Form (3NF)

Aston University
BIRMINGHAM UK

- All relations must comply with 2NF
- No transitive functional dependencies are allowed

Student (id, fname, sname)

id	fname	sname
5	Stephen	Garrett
7	Lucy	Warren
9	Myra	Gordon
4	Neil	Davis
6	David	Binder

Relatives (nextofkin, nextofkin_tel, build_id, stu_id)

nextofkin	nextofkin_tel	build_id	stu_id
M. Garrett	255-0608	10	5
K. Garrett	739-6134	10	5
B. Warren	999-6173	20	7
C. Gordon	248-9612	30	9
L. Davis	287-6479	40	4
A. Binder	644-2297	50	6

Building (build_id, building, postcode)

build_id	building	postcode
10	Wembley	SY23 5DD
20	Jendale	HU7 4ET
30	Ifton	SY11 3AF
40	Vega	WD23 4TW
50	Maffon	CF46 6HE

Discussion

Aston University
BIRMINGHAM UK

Normal forms solve different problems:

- **1NF**: only atomic domains, no repeated groups
- **2NF**: no partial functional dependencies
- **3NF**: no transitive functional dependencies

Points to remember:

- Higher normal forms require more but smaller tables
- More tables can bring higher cost of maintenance
- It is important to meet the needs of applications

Summary

You can now better discuss the following topics:

- Database anomalies and when they occur
- The SQL Join command and schema decomposition
- Functional dependencies and normalisation